

第 8 章

密码协议

8.1 协议的基本概念

在现实生活中，人们对协议并不陌生，人们都在自觉或不自觉地使用各种协议。例如，在处理国际事务时，国家政府之间通常要遵守某种协议；在法律上，当事人之间常常要按照规定的法律程序去处理纠纷；在打扑克、电话订货、投票或到银行存/取款时，都要遵守特定的协议。由于人们能够熟练地使用这些协议来有效地完成所要做的事情，所以很少有人去深入地考虑它们。

协议（protocol）指两个或两个以上的参与者为完成某项特定的任务而采取的一系列步骤。这个定义包含 3 层含义：第一，协议自始至终是有序的过程，每一步骤必须依次执行。在前一步没有执行完之前，后面的步骤不可能执行。第二，协议至少需要两个参与者。一个人可以通过执行一系列的步骤来完成某项任务，但它不构成协议。第三，通过执行协议必须能够完成某项任务。即使某些东西看似协议，但没有完成任何任务，也不能称为协议，只不过是浪费时间的空操作。

在讨论之前，首先对协议的参与者做表 8-1 所示的定义。

表 8-1 协议中可能的参与者及其作用

协议的参与者	其在协议中所发挥的作用
Alice	在所有协议中，她是第一参与者
Bob	在所有的协议中，他是第二参与者
Carol	在三方或四方协议中，他是参与者之一
Dave	在三方或四方协议中，他是参与者之一
Eve	窃听者
Mallory	恶意的主动攻击者
Trent	可信赖的仲裁者
Walter	监察官，他将在某些协议中保护 Alice 和 Bob
Peggy	证明者
Victor	验证者

8.1.1 仲裁协议

仲裁者（Arbitrator）是某个公正的第三方。在执行协议的过程中，其他各方均信赖

他。“公正”意味着仲裁者对参与协议的任何一方没有偏向，而“可信赖”意味着参与协议的所有人均认为他所说的话都是真的，他所做的事都是正确的，并且他将完成协议赋予他的任务。仲裁者能够帮助两个互不信赖的实体完成协议，如图 8-1 所示。

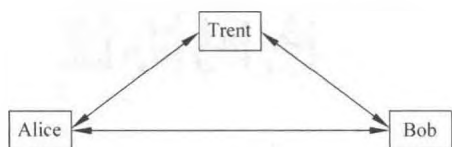


图 8-1 仲裁协议

在现实生活中，律师常常被认为是仲裁者。例如，Alice 要卖汽车给陌生人 Bob，而 Bob 想用支票付账。在 Alice 将车交给 Bob 之前，他必须查清支票的真伪。同样，Bob 也不相信 Alice，在没有获得车主权之前，也不愿将支票交给 Alice。

这时，就需要一个为双方信赖的律师来帮助他们完成交易。Alice 和 Bob 可以通过执行以下协议来确保彼此不受欺骗：

(1) Alice 将车主权和钥匙交给律师。

(2) Bob 将支票交给 Alice。

(3) Alice 在银行兑现支票。

(4) 在规定的时间内，若证明支票是真的，律师将车主权和钥匙交给 Bob；若证明支票是假的，Alice 将向律师提供确切的证据，此后律师将车主权和钥匙交还给 Alice。

在这一协议中，Alice 相信在她弄清支票的真伪之前律师不会将车主权交给 Bob，一旦发现支票有假，律师还会将车主权归还她；Bob 也相信律师在支票兑现后，将把车主权和钥匙交给他。在协议中，律师只起担保代理作用，他并不关心支票的真伪。

银行也可以充当仲裁人的角色。通过执行以下协议，Bob 可以从 Alice 手中买到车：

(1) Bob 开一张支票并将其交给银行。

(2) 在验明 Bob 的钱足以支付支票上的数目后，银行将保付支票交给 Bob。

(3) Alice 将车主权和钥匙交给 Bob。

(4) Bob 将保付支票交给 Alice。

(5) Alice 兑现支票。

这个协议是有效的，因为 Alice 相信银行开具的证明。同时，Bob 也相信银行不会将他的钱用于其他不正当的场合。

然而，在计算机领域中，让计算机充当仲裁人时，会遇到如下一些新的问题：

- 在计算机网络中，彼此互不信赖的通信双方进行通信时，也需要某台计算机充当仲裁者。但是，由于计算机网络的复杂性，使得互相怀疑的通信双方很可能也怀疑作为仲裁者的计算机。
- 在计算机网络中，要设立一个仲裁者，就要像聘请律师一样付出一定的费用。然而，在网络环境下，没有人愿意承担这种额外的开销。
- 当协议中引入仲裁者时，会增加时延。
- 由于仲裁者需要对每一次会话加以处理，它有可能成为系统的瓶颈。在实现时，增加仲裁者的数目可能会缓解这个问题，但是这会增加系统的造价。
- 在网络中，由于每个人都必须信赖仲裁者，因此它也就成为攻击者攻击的焦点。
- 在具有仲裁的协议中，仲裁人的角色由 Trent 来担任。

8.1.2 裁决协议

由于在协议中引入仲裁人会增加系统的造价，所以在实际应用中，通常引入另外一种协议，称为裁决协议。只有发生纠纷时，裁决人才执行此协议；而无纠纷发生时，并不需要裁决人的参与，如图 8-2 所示。

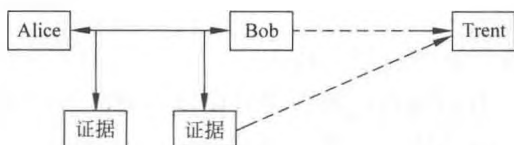


图 8-2 裁决协议

与仲裁人一样，裁决人（adjudicator）也是一个公正的、可信赖的第三方。他不像仲裁者一样直接参与协议。例如，法官是职业裁决人。Alice 和 Bob 在签署合同时，并不需要法官的参与。但是，当他们之间发生纠纷时，就需要法官来裁决。

合同签署协议可以规范地做如下表述：

无仲裁的子协议：

- (1) Alice 和 Bob 协商协议的条款。
- (2) Alice 签署这个合同。
- (3) Bob 签署这个合同。

裁决子协议：

- (1) Alice 和 Bob 出现在法官面前。
- (2) Alice 向法官提供她的证据。
- (3) Bob 向法官提供他的证据。
- (4) 法官根据双方提供的证据进行裁决。

在计算机网络环境下，也有裁决协议。这些协议建立在各方均是诚实的基础之上。但是，当有人怀疑发生欺骗时，可信赖的第三方就可以根据所存在的某个数据项判定是否存在欺骗。一个好的裁决协议应该能够确定欺骗者的身份。注意，裁决协议只能检测欺骗是否存在，而不能防止欺骗的发生。

8.1.3 自动执行协议

自动执行协议是最好的协议。协议本身就保证了公平性，如图 8-3 所示。这种协议不需要仲裁者的参与，也不需要裁决者来解决争端。如果协议中的一方试图欺骗另一方，那么另一方会立刻检测到该欺骗的发生，并停止执行协议。



图 8-3 自动执行协议

今天，人们越来越多地使用计算机网络进行交流。计算机能够代替人们完成要做的事情，但是它必须按照事先设计的协议来执行。人可以对新的环境做出相应的反应，而

计算机却不能。在这一点上，计算机几乎无灵活性可言。

因此，协议应该对所要完成的某项任务的过程加以抽象。无论对 PC 还是对 VAX 机来说，所采用的通信协议都是相同的。这种抽象不仅可以大大提高协议的适应性，也可以使人们十分容易地辨别协议的优劣。协议不仅应该具有很高的运行效率，而且应该具有行为上的完整性。在设计协议时，应该考虑到完成某项任务时可能发生的各种情况，并对其做出相应的反应。

因此，一个好的协议应该具有以下特点：

- (1) 协议涉及的每一方必须事先知道此协议以及要执行的所有步骤。
- (2) 协议涉及的每一方必须同意遵守协议。
- (3) 协议必须是非模糊的。对协议的每一步都必须确切定义，力求做到避免产生误解。
- (4) 协议必须是完整的。对每一种可能发生的情况都要做出反应。
- (5) 每一步操作要么是由一方或多方进行计算，要么是在各方之间进行消息传递，二者必居其一。

许多面对面的协议依赖于人出场来保证真实性和安全性。例如，购物时，不可能将支票交给陌生人；与他人玩扑克时，必须保证亲眼看到他洗牌和发牌。然而，当通过计算机与远端的用户进行交流时，真实性和安全性便无法保证。实际上，不仅难以保证使用计算机网络的所有用户都是诚实的，而且也难以保证计算机网络的管理者和设计者都是诚实的。只有通过使用规范化的协议，才可以有效地防止不诚实的用户对网络实施的各种攻击。

从上面的讨论可知，计算机网络中使用的好的通信协议，不仅应该具有有效性、公平性和完整性，而且应该具有足够高的安全性。通常把具有安全性功能的协议称为安全协议。安全协议的设计必须采用密码技术。因此，有时也将安全协议称做密码协议。

密码协议与许多通信协议的显著区别在于它使用了密码技术。在进行密码协议的设计时，常常要用到某些密码算法。密码协议所涉及的各方可能是相互信赖的，也可能彼此互不信任。当成千上万的用户在网络上进行信息交互时，会给网络带来严重的安全问题。例如，非法用户不必对网络上传输的信息解密，就可能利用网络协议自身存在的安全缺陷，获取合法用户的某些机密信息（如用户口令、密钥、用户身份号等），从而冒充合法用户无偿使用网络资源，或窃取网络数据库中的秘密用户文档。因此，设计安全、有效的通信协议，是密码学和通信领域中一个十分重要的研究课题。密码协议的目标不仅仅是实现信息的加密传输，而更重要的是为了解决通信网的安全问题。参与通信协议的各方可能想分享部分秘密来计算某个值、生成某个随机序列、向对方表明自己的身份，或签订某个合同。在协议中采用密码技术，是防止或检测非法用户对网络进行窃听和欺骗攻击的关键技术措施。所谓协议是安全的，意味着非法用户不可能从协议中获得比协议自身所体现的更多的、有用的信息。

在后面几节里，将要讨论许多密码协议。其中，有些协议是不安全的，可能会导致参与协议的一方欺骗另一方。还有一些协议窃听者可以攻破或者能从中获取某些秘密信息。造成协议失败的原因有多种，最主要的是因为协议的设计者对安全需求的定义研究

得不够透彻，并且对设计出来的协议缺乏足够的安全性分析。正像密码算法的设计一样，要证明协议的不安全性要比证明其安全性容易得多。

8.2 安全协议分类及基本密码协议

迄今，尚未有人对安全协议进行过详细的分类。其实，将密码协议进行严格分类并非易事。从不同的角度出发，就有不同的分类方法。例如，根据安全协议的功能，可以将其分为认证协议、密钥建立（交换、分配）协议、认证的密钥建立（交换、分配）协议；根据 ISO 的 7 层参考模型，又可以将其分成高层协议和低层协议；按照协议中所采用的密码算法的种类，又可以分成双钥（或公钥）协议、单钥协议或混合协议等。作者认为，比较合理的分类方法是应该按照密码协议的功能来分类，而不管协议具体采用何种密码技术。因此，把密码协议分成以下 3 类：

（1）密钥建立协议（Key Establishment Protocol）：通信双方建立共享秘密。

（2）认证协议（Authentication Protocol）：一个实体向与其通信的另一个实体提供其身份的可信性。

（3）认证的密钥建立协议（Authenticated Key Establishment Protocol）：与另一身份已被或可被证实的实体之间建立共享秘密。

下面对这 3 类协议进行详细讨论。

8.2.1 密钥建立协议

密钥建立协议可在两个或多个实体之间建立共享的秘密，该共享秘密可用于数据加密，通常用做建立一次通信时的会话密钥。下面将主要讨论在两个实体之间建立共享秘密的协议问题。它可以采用单钥、双钥技术实现，有时也要借助于可信赖第三者的参与。可以将其扩展到多方共享密钥，如会议密钥建立，但随着参与方增多，协议会迅速变得很复杂。

在保密通信中，通常对每次会话都采用不同的密钥进行加密。因为这个密钥只用于对某个特定的通信会话进行加密，所以被称为会话密钥。会话密钥只在通信的持续范围内有效，当通信结束后，会话密钥会被清除。如何将这些会话密钥分发到会话者的手中，是本节要讨论的问题。

1. 采用单钥体制的密钥建立协议

密钥建立协议主要可分为密钥传输协议和密钥协商协议，前者是将一个实体建立或收到的密钥安全传送给另一个实体，而后者是由双方（或多方）共同提供信息建立起共享密钥，任何一方都不起决定作用。其他如密钥更新、密钥推导、密钥预分配、动态密钥建立机制等都可由上述两种基本密钥建立协议变化得出。

可信赖服务器（或可信赖第三方、认证服务器、密钥分配中心 KDC、密钥传递中心 KTC、证书发行机构 CA 等）可以在初始化建立阶段、在线实时通信或两者都有的情

况下参与密钥分配。

这类协议假设网络用户 Alice 和 Bob 各自都与密钥分配中心 KDC（在协议中扮演 Trent 的角色）共享一个密钥。这些密钥在协议开始之前必须已经分发到位。在下面的讨论中，并不关心如何分发这些共享密钥，仅假设它们早已分发到位，而且 Mallory 对它们一无所知。协议描述如表 8-2 所示。

表 8-2 采用单钥体制的密钥建立协议

密钥建立协议——采用单钥体制

- (1) Alice 呼叫 Trent，并请求得到与 Bob 通信的会话密钥。
- (2) Trent 生成一个随机会话密钥，并做两次加密：一次是采用 Alice 的密钥，另一次是采用 Bob 的密钥。Trent 将两次加密的结果都发送给 Alice。
- (3) Alice 采用共享密钥对属于她的密文解密，得到会话密钥。
- (4) Alice 将属于 Bob 的那项密文发送给他。
- (5) Bob 对收到的密文采用共享密钥解密，得到会话密钥。
- (6) Alice 和 Bob 均采用该会话密钥进行安全通信。

此协议的安全性，完全依赖于 Trent 的安全性。Trent 可能是一个可信赖的通信实体，更可能是一个可信赖的计算机程序。如果 Mallory 买通了 Trent，那么整个网络的机密就会泄漏。由于掌握了所有用户与 Trent 共享的密钥，Mallory 就可以阅读所有过去截获的消息和将来的通信业务。他只需对通信线路搭线，就可以窃听所有加密的消息流。

上述协议存在的另外一个问题是：Trent 可能成为影响系统性能的瓶颈，因为每次进行密钥交换时，都需要 Trent 的参与。如果 Trent 出现问题，就会影响整个系统的正常工作。

2. 采用双钥体制的密钥建立协议

在实际应用中，Bob 和 Alice 常采用双钥体制来建立某个会话密钥，此后采用此会话密钥对数据进行加密。在某些具体实现方案中，Bob 和 Alice 的公钥被可信赖的第三方签名后，存放在某个数据库中。这就使密钥建立协议变得更加简单。即使 Alice 从未听说过 Bob，她也能与其建立安全的通信联系。协议描述如表 8-3 所示。

表 8-3 采用双钥体制的密钥建立协议

密钥建立协议——采用双钥体制

- (1) Alice 从数据库中得到 Bob 的公钥。
- (2) Alice 生成一个随机的会话密钥，采用 Bob 的公钥加密后，发送给 Bob。
- (3) Bob 采用其私钥对 Alice 的消息进行解密。
- (4) Bob 和 Alice 均采用同一会话密钥对通信过程中的消息加密。

3. 中间人攻击（Men-in-the-middle Attack）

当 Eve 找不到比攻破双钥算法或对密文实施唯密文攻击更好的方法时，Mallory 的攻击显得更加危险。他不仅能够窃听 Alice 和 Bob 之间交换的消息，而且能够篡改消息，

删除消息，甚至生成全新的消息。当 Bob 与 Alice 会话时，Mallory 可以冒充 Bob；当 Alice 与 Bob 会话时，Mallory 可以冒充 Alice。这就是中间人攻击。Mallory 对协议的攻击如下：

(1) Alice 发送她的公钥给 Bob。Mallory 截获这一公钥，并将他自己的公钥发送给 Bob。

(2) Bob 发送他的公钥给 Alice。Mallory 截获这一公钥，并将他自己的公钥发送给 Alice。

(3) 当 Alice 采用“Bob”的公钥对消息加密并发送给 Bob 时，Mallory 会截获到它。由于这条消息实际上是采用了 Mallory 的公钥进行加密，因此他可以采用其私钥进行解密，并采用 Bob 的公钥对消息重新加密后发送给 Bob。

(4) 当 Bob 采用“Alice”的公钥对消息加密并发送给 Alice 时，Mallory 会截获到它。由于这条消息实际上是采用了 Mallory 的公钥进行加密，因此他可以采用其私钥进行解密，并采用 Alice 的公钥对消息重新加密后发送给 Alice。

即使 Alice 和 Bob 的公钥存放在数据库中，这一攻击仍然有效。Mallory 可以截获 Alice 的数据库查询指令，并用其公钥替换 Bob 的公钥。同样，他也可以截获 Bob 的数据库查询指令并用其公钥替代 Alice 的公钥。更为严重的是，Mallory 可以进入数据库中，将 Alice 和 Bob 的公钥均替换成他自己的公钥。此后，他只须等待 Alice 与 Bob 会话，截获并篡改消息。

中间人攻击之所以起作用，是因为 Alice 和 Bob 没有办法来验证他们正在与另一方会话。假设 Mallory 没有产生任何可以察觉的网络时延，那么 Alice 和 Bob 不会知道有人正在他们之间阅读所有的秘密信息。

4. 联锁协议

联锁协议 (Interlock Protocol) 由 R. Rivest 和 A. Shamir 设计[Rivest 等 1984]，该协议能够有效地抵抗中间人攻击。协议描述如表 8-4 所示。

表 8-4 联锁协议

密钥建立协议——联锁协议

- (1) Alice 发送她的公钥给 Bob。
 - (2) Bob 发送他的公钥给 Alice。
 - (3) Alice 用 Bob 的公钥对消息加密。此后，她将一半密文发送给 Bob。
 - (4) Bob 用 Alice 的公钥对消息加密。此后，他将一半密文发送给 Alice。
 - (5) Alice 发送另一半密文给 Bob。
 - (6) Bob 将 Alice 的两半密文组合在一起，并采用其私钥解密。Bob 发送他的另一半密文给 Alice。
 - (7) Alice 将 Bob 的两半密文组合在一起，并采用其私钥解密。
-

这个协议最重要的一点是：当仅获得一半而没有获得另一半密文时，这些数据对攻击者来说毫无用处，因为攻击者无法解密。在第 (6) 步以前，Bob 不可能读到 Alice 的任何一部分消息。在第 (7) 步以前，Alice 也不可能读到 Bob 的任何一部分消息。要做到这一点，有以下几种方法：

(1) 如果加密算法是一个分组加密算法，每一半消息可以是输出的密文分组的一半。

(2) 对消息解密可能要依赖于某个初始化矢量，该初始化矢量可以作为消息的第二半发送给对方。

(3) 发送的第一半消息可以是加密消息的单向杂凑函数值，而加密的消息本身可以作为消息的另一半。

现在来分析 Mallory 是如何对此协议进行攻击的。Mallory 仍然可以在第(1)和(2)步中用他的公钥来替代 Alice 和 Bob 的公钥。但是现在，当他在第(3)步中截获到 Alice 的一半消息时，他既不能对其解密，也不能用 Bob 的公钥重新加密。他必须产生一个全新的消息，并将其一半发送给 Bob。当他在第(4)步中截获 Bob 发给 Alice 的一半消息时，他会遇到相同的问题，即：他既不能对其解密，也不能用 Alice 的公钥重新加密。他必须产生一个全新的消息，并将其一半发送给 Alice。当 Mallory 在第(5)和(6)步中截获到真正的第二半消息时，对他来说为时已晚，以至于来不及对前面伪造的消息进行修改。Alice 和 Bob 会发现这种攻击，因为他们谈话的内容与伪造的消息有可能完全不同。

Mallory 也可以不采用这种攻击方法。如果他非常了解 Alice 和 Bob，他就可以假冒其中一人与另一人通话，而他们绝不会想到正在受骗。但这样做肯定要比充当中间人更难。

5. 采用数字签名的密钥交换

在会话密钥交换协议中采用数字签名技术，可以有效地防止中间人攻击。Trent 是一个可信赖的实体，他对 Alice 和 Bob 的公钥做数字签名。签名的公钥中包含一个所有权证书。当 Alice 和 Bob 收到此签名公钥时，他们每人均可以通过验证 Trent 的签名来确定公钥的合法性，因为 Mallory 无法伪造 Trent 的签名。

这样一来，Mallory 的攻击就变得十分困难：他不能实施假冒攻击，因为他既不知道 Alice 的私钥，也不知道 Bob 的私钥；他也不能实施中间人攻击，因为他不能伪造 Trent 的签名。即使他能够从 Trent 那里获得一个签名公钥，Alice 和 Bob 也很容易发现该公钥属于他。Mallory 能做的只有窃听往来的加密报文，或者干扰通信线路，阻止 Alice 与 Bob 会话。

这一协议中引入了 Trent 这个角色。然而，密钥分配中心(KDC)遭到攻击、泄漏秘密的风险要比第一个协议小得多。如果 Mallory 侵入了 KDC，他能够得到的仅仅是 Trent 的私钥。Mallory 可以利用这一私钥给用户签发新的公钥，但不能用其解密任何会话密钥或阅读任何报文。要想阅读报文，Mallory 必须假冒某个合法网络用户，并欺骗其他合法用户采用 Mallory 的公钥对报文加密。

一旦 Mallory 获得了 Trent 的私钥，他就能够对协议发起中间人攻击。他采用 Trent 的私钥对一些伪造的公钥签名。此后，他或者将数据库中 Alice 和 Bob 的真正公钥换掉，或者截获用户的数据库访问请求，并用伪造的公钥响应该请求。这样，他就可以成功地发起中间人攻击，并阅读他人的通信。

这一攻击是奏效的，但是前提条件是 Mallory 必须获得 Trent 的私钥，并对加密消息

进行截获或篡改。在某些网络环境下,这样做显然要比在两个用户之间实施被动的窃听攻击难得多。对于像无线网络这样的广播信道来说,尽管可以对整个网络实施干扰破坏,但是要想用一个消息取代另一个消息几乎是不可能的。对于计算机网络来说,这种攻击要容易得多,而且随着技术的发展,这种攻击变得越来越容易。考虑到现存的 IP 欺骗、路由器攻击等,主动攻击并不意味着非要对加密的报文解密,也不只限于充当中间人。此外,还有许多更加复杂的攻击需要研究。

6. 密钥和消息传输

Alice 和 Bob 不必先完成密钥交换协议,再进行信息交换。在下面的协议中, Alice 在事先没有执行密钥交换协议的情况下,将消息 M 发送给 Bob,协议描述如表 8-5 所示。

表 8-5 采用单钥与双钥混合体制的密钥建立协议

密钥建立协议——单钥与双钥体制混合协议

- (1) Alice 生成一随机数作为会话密钥,并用其对消息 M 加密: $E_K(M)$ 。
- (2) Alice 从数据库中得到 Bob 的公钥。
- (3) Alice 用 Bob 的公钥对会话密钥加密: $E_B(K)$ 。
- (4) Alice 将加密的消息和会话密钥发送给 Bob: $E_K(M), E_B(K)$ 。
- *为了提高协议的安全性以对付中间人攻击, Alice 可以对这条消息签名。
- (5) Bob 采用其私钥对 Alice 的会话密钥解密。
- (6) Bob 采用这一会话密钥对 Alice 的消息解密。

这一协议中既采用了双钥体制,也采用了单钥体制。这种混合协议在通信系统中经常用到。一些协议还常常将数字签名、时戳和其他密码技术结合在一起。

7. 密钥和消息广播

在实际应用中, Alice 也可能将消息同时发送给几个人。在下面的例子中, Alice 将加密的消息同时发送给 Bob、Carol 和 Dave:

- (1) Alice 生成一随机数作为会话密钥 K ,并用其对消息 M 加密: $E_K(M)$ 。
- (2) Alice 从数据库中得到 Bob、Carol 和 Dave 的公钥。
- (3) Alice 分别采用 Bob、Carol 和 Dave 的公钥对 K 加密: $E_B(K), E_C(K), E_D(K)$ 。
- (4) Alice 广播加密的消息和所有加密的密钥,将它们传送给要接收的人。
- (5) 仅有 Bob、Carol 和 Dave 能采用各自的私钥解密求出会话密钥 K 。
- (6) 仅有 Bob、Carol 和 Dave 能采用此会话密钥 K 对消息解密求出 M 。

这一协议可以在存储转发网络上实现。中央服务器可以将 Alice 的消息和各自的加密密钥一起转发给他们。服务器不必是安全的和可信赖的,因为它不能解密任何消息。

8. Diffie-Hellman 密钥交换协议

Diffie-Hellman 算法是在 1976 年提出的[Diffie 等 1976a]。它是第一个双钥算法,其安全性基于在有限域上计算离散对数的难度。Diffie-Hellman 协议可以用做密钥交换,

Alice 和 Bob 可以采用这个算法共享一个秘密的会话密钥，但不能采用它来对消息进行加密或解密。

该协议的原理十分简单。首先，Alice 和 Bob 约定两个大的素数， n 和 g ，使得 g 是群 $\langle 0, \dots, n-1 \rangle$ 上的本原元。这两个整数不必保密，Alice 和 Bob 可以通过不安全的信道传递它们。即使许多用户知道这两个数，也没有关系。协议描述如表 8-6 所示。

表 8-6 Diffie-Hellman 密钥交换协议

Diffie-Hellman 密钥交换协议

- (1) Alice 选择一个随机的大整数 x ，并向 Bob 发送以下消息： $X=g^x \bmod n$ 。
- (2) Bob 选择一个随机的大整数 y ，并向 Alice 发送以下消息： $Y=g^y \bmod n$ 。
- (3) Alice 计算： $K=Y^x \bmod n$ 。
- (4) Bob 计算： $K'=X^y \bmod n$ 。

至此， K 和 K' 均等于 $g^{xy} \bmod n$ 。任何搭线窃听的人均不能计算得到该值，除非攻击者能够计算离散对数来得到 x 和 y ，否则它们无法获得该密钥。所以， K 可以被 Alice 和 Bob 用做会话密钥。

g 和 n 的选择对于系统的安全性有着根本的影响。 $(n-1)/2$ 应该是素数[Pohlig 等 1978]，最重要的是 n 应该足够大。这样，系统的安全性就基于分解与 n 具有同样长度的数的难度。可以选择 g 使得 g 是群 $\langle 0, \dots, n-1 \rangle$ 上的本原元，也可以选择最小的 g （通常只有 1 位数）。实际上， g 不一定必须是本原元，只要用它能够生成乘群 $\langle 0, \dots, n-1 \rangle$ 的一个大子群即可。

Diffie-Hellman 的密钥交换协议可以很容易地扩展到多个用户的情况，该算法也可以从乘群上扩展到交换环上 [Pohlig 等 1978]。Z. Shmuley 和 K. McCurley 提出了该算法的另外一种形式，其中模是一个大合数 [Shmuley 1985; McCurley 1988]。V. S. Miller 和 N. Koblitz 将这一算法扩展到椭圆曲线上 [Miller 1985; Koblitz 1987]。T. ElGamal 利用这一算法的思想设计了一种加密和数字签名算法（见 7.4 节）。

这一算法也可以在伽罗华域 $GF(2^k)$ 上实现 [Shmuley 1985; McCurley 1988]。由于在伽罗华域上进行指数运算很快，所以现实中的许多设计均采用这一方法。同样，在对算法进行密码分析时运算速度也会很快，因此对我们来说，重要的是应该细心选择一个足够大的域，以保证系统的安全性。

8.2.2 认证建立协议

如第 6 章所述，认证包含消息认证、数据源认证和实体认证（身份识别），用以防止欺骗、伪装等攻击。有关技术算法前面已经介绍，这里讨论实现认证的各种协议。

当 Alice 登录到某个主机（或者某个自动取款机、电话银行系统或其他任何类型的终端）时，主机如何知道她是谁呢？主机怎么才能知道她不是 Eve 假冒 Alice 的身份？传统的方法是采用口令来解决这个问题。Alice 输入她的口令，主机确认口令是正确的。Alice 和主机都知道这一秘密。每次登录时，主机都要求 Alice 输入她的口令。

1. 采用单向函数的认证协议

R. Needham 和 M. Guy 等指出：在对 Alice 进行认证时，主机无须知道其口令。它只需能够辨别 Alice 提交的口令是否有效。这很容易通过采用单向函数来实现。主机不必存储 Alice 的口令，它只需存储该口令的单向函数值。协议描述如表 8-7 所示。

表 8-7 采用单向函数的认证协议

采用单向函数的认证协议
(1) Alice 向主机发送她的口令。
(2) 主机计算该口令的单向函数值。
(3) 主机将计算得到的单向函数值与预先存储的值进行比较。

由于主机不需要再存储各用户的有效口令表，减轻了攻击者侵入主机和窃取口令清单的威胁。攻击者窃取口令的单向函数值将毫无用处，因为他不可能从单向函数值中反向推出用户的口令。

2. 字典攻击和掺杂

一个采用单向函数加密的口令文件仍然易遭受攻击。Mallory 可以编制 100 万个最常用的口令，然后用单向函数对所有这些口令加密并存储密文。若每个口令为 8 字节，那么密文不会超过 8MB。此后，Mallory 可以窃取某个加密的口令文件，并与他存储的密文相比较，看有哪些密文重合。这种方法被称为字典攻击。事实证明，这种攻击方法十分有效。

掺杂是一种使字典攻击变得更加困难的方法。掺杂是一个伪随机序列，常常将其与口令级联后再采用单向函数加密。此后，将掺杂值和密文一起存储于主机的数据库中。如果掺杂值的空间足够大，就会大大削弱字典攻击的成功概率，因为 Mallory 必须对每个可能的掺杂值加密，生成一个单向杂凑值。

这里需要弄清的一点是，当 Mallory 试图攻破某个人的口令时，他必须试着对字典中的每个口令进行加密，而不是针对所有的口令进行大量的预计算。

许多 UNIX 系统仅采用 12b 的掺杂。即便如此，Daniel Klein 通过一个口令揣测程序，在一周之内便可以破译任何一台主机上 40% 的口令 [Klein 1990]。David Feldmeier 和 Philip Karn 收集了大约 73.2 万个常用的口令，每个口令均与 4096 个可能的 Salt 值相级联。他们估计采用这一口令表，对任意一台给定的主机，有 30% 的口令可以被攻破。

然而，掺杂并不是万灵药，仅靠增加掺杂比特的数目并不会解决所有的问题。掺杂仅能抗击对口令文件的一般字典攻击，而不能抗击对单一口令的预定攻击。它可以保护人们多台计算机上使用同一口令，却不能使选择的坏口令变得更安全。

3. SKEY 认证程序

SKEY 是一个认证程序，它的安全性取决于所采用的单向函数。它的工作原理如下：开始时，Alice 输入一个随机数 R 。计算机计算 $f(R), f(f(R)), f(f(f(R)))$ 等 100 次，

将其记为 $x_1, x_2, x_3, \dots, x_{100}$ 。之后，计算机打印出这些数的清单，并安全保存。同时计算机也将 x_{101} 和 Alice 的姓名一起存放在某个登录数据库中。

在 Alice 首次登录时，输入其姓名和 x_{100} 。计算机计算 $f(x_{100})$ ，并将其与存储在数据库中的值 x_{101} 加以比较；如果它们相等，Alice 就通过认证。然后，计算机用 x_{100} 将数据库中的 x_{101} 取代；Alice 也将 x_{100} 从她的清单中去掉。

每次登录时，Alice 输入清单中最后一个未被去掉的数 x_i 。计算机计算 $f(x_i)$ ，并将其与存储在数据库中的 x_{i+1} 进行比较。由于每个数仅用一次，而且函数是单向的，Eve 不能得到任何有用的信息。同样，数据库对于攻击者来说仍然有用。当然，当 Alice 用完了清单中的数时，她必须重新对该系统进行初始化。

4. 采用双钥体制的认证

即使采用了掺杂，表 8-7 描述的协议仍然存在严重的安全问题。当 Alice 向主机发送口令时，接入其数据通道的任何人都可以阅读到此口令。她也许通过某个复杂的传输通道访问她的主机，而这个通道可能要经过 4 个工业集团、3 个国家和两所大学。Eve 可能就正在其中的任何一个节点上来窃听 Alice 的登录序列。如果 Eve 能够接入到主机的处理器内存，他就会抢在主机对口令做杂凑运算之前看到该口令。

采用双钥密码体制可以解决这个问题。主机保留每个用户的公钥文件；所有的用户保留他们各自的私钥。协议描述如表 8-8 所示。

表 8-8 采用双钥体制的认证协议

认证协议——采用双钥体制
(1) 主机向 Alice 发送一随机数。
(2) Alice 用其私钥对此随机数加密，并将密文连同其姓名一起发送给主机。
(3) 主机在它的数据库中搜索 Alice 的公钥，并采用此公钥对收到的密文解密。
(4) 如果解密得到的消息与主机首次发给 Alice 的数值相等，主机就允许 Alice 对系统进行访问。

由于无人能够访问 Alice 的私钥，所以也就无人能够假冒 Alice。最重要的是，Alice 永远不会将其私钥发给主机。即使 Eve 可以窃听到 Alice 与主机之间的会话，他也不能获得可以用来推出私钥并冒充 Alice 的任何信息。

Alice 的私钥不但很长，而且难以记忆。它可能由用户的硬件产生，也可能由用户的软件产生，只要求 Alice 拥有一个可信赖的智能终端，并不要求主机必须是安全的，也不要求通信通道必须是安全的。

实际中，对数据串的选择必须十分谨慎。不仅是因为存在不可信赖的第三方问题，而且还存在其他类型的有效攻击。因此，安全的身份证明协议常采用以下更加复杂的形式：

- (1) Alice 基于某些随机数和其私钥进行加密运算，并将结果送给主机。
- (2) 主机向 Alice 发送另外一个随机数。
- (3) Alice 基于随机数（她自己生成的一些随机数和收到来自主机的某个随机数）以及她的私钥进行计算，并将结果发给主机。

(4) 主机采用 Alice 的公钥对收到的数值进行解密, 看 Alice 是否知道她的私钥。

(5) 若 Alice 确实知道她的私钥, 那么她的身份就得以确认。

如果 Alice 并不信赖主机, 那么她就要求主机以同样的方式来证明其身份。

协议的第 (1) 步看起来似乎没有必要或者令人费解, 然而却是抵抗攻击者对协议攻击所必需的步骤 [Lamport 1981]。

5. 采用联锁协议的双向认证

Alice 和 Bob 是两个想要进行相互认证的用户。每个人都有一个对方已知的口令: Alice 具有 P_A , Bob 具有 P_B 。下面是一个不安全的协议:

- (1) Alice 和 Bob 相互交换公钥。
- (2) Alice 用 Bob 的公钥对 P_A 加密, 并将结果送给 Bob。
- (3) Bob 用 Alice 的公钥对 P_B 加密, 并将结果送给 Alice。
- (4) Alice 对在 (3) 中收到的消息解密, 并验证其是否正确。
- (5) Bob 对在 (2) 中收到的消息解密, 并验证其是否正确。

Mallory 可以对上面的协议成功地实施中间人攻击。攻击方法如表 8-9 所示。

表 8-9 中间人攻击

中间人攻击

- (1) Alice 和 Bob 相互交换公钥。Mallory 可以截获通信双方的公钥 P_A 和 P_B , 他用自己的公钥替换掉 Bob 的公钥, 并将其发送给 Alice。然后, 他用自己的公钥替换掉 Alice 的公钥, 并将其发送给 Bob。
- (2) Alice 用 “Bob” 的公钥对 P_A 加密, 并将其发送给 Bob。Mallory 可以截获这一消息, 并用其私钥解密求出 P_A , 再用 Bob 的公钥重新对 P_A 加密, 并将结果发送给 Bob。
- (3) Bob 用 “Alice” 的公钥对 P_B 加密, 并将其发送给 Alice。Mallory 可以截获这一消息, 并用其私钥解密求出 P_B , 再用 Alice 的公钥重新对 P_B 加密, 并将结果发送发给 Alice。
- (4) Alice 解密求出 P_B , 并验证其是否正确。
- (5) Bob 解密求出 P_A , 并验证其是否正确。

在 Alice 和 Bob 看来, 此认证过程并没有什么不妥。然而, 对于 Mallory 来说, 他可以获得通信双方的口令 P_A 和 P_B 。

D. Davies 和 W. Price 描述了如何利用联锁协议挫败这一攻击 [Davies 等 1989]。S. Bellovin 和 M. Merritt 讨论了攻击这一协议的方法 [Bellovin 等 1994]。如果 Alice 是一个用户, 而 Bob 是一个主机, Mallory 可以假装成 Bob, 与 Alice 一起完成协议的开头几步, 然后断掉与 Alice 的连接。Mallory 通过模拟线路噪声或网络故障来欺骗对方, 结果是 Mallory 获得了 Alice 的口令。此后, 他与 Bob 建立连接并完成协议, 最终获得 Bob 的口令。

协议可以做进一步的修改: 假设用户的口令比主机的口令更加敏感, 此时 Bob 先于 Alice 给出他的口令。修改后的协议可能遭受更加复杂的攻击 [Bellovin 等 1994]。

6. SKID 身份识别协议

SKID2 和 SKID3 是采用单钥体制构造的身份识别协议，它们是为 RACE 的 RIPE 计划而开发的[RACE 1992]。它们采用了消息认证码 MAC 来提供安全性，并且假设 Alice 和 Bob 共享一个密钥 K 。

SKID2 允许 Bob 向 Alice 提供其身份。SKID3 提供了 Alice 和 Bob 之间的双向认证。协议如表 8-10 所示。

表 8-10 SKID2、SKID3 身份识别协议

SKID2 身份识别协议

- (1) Alice 选择随机数 R_A （在 RIPE 文件中规定其为 64bit），并将其发送给 Bob。
- (2) Bob 选择随机数 R_B （在 RIPE 文件中规定其为 64bit），并发送给 Alice 消息： $R_B, H_K(R_A, R_B, B)$ 。其中， H_K 是消息认证码(MAC)，在 RIPE 文件中建议 MAC 采用 RIPE-MAC 函数， B 是 Bob 的姓名识别符。
- (3) Alice 计算 $H_K(R_A, R_B, B)$ ，并将其与收到的来自 Bob 的值进行比较。如果两值相等，那么 Alice 知道她正在与 Bob 通信。

SKID3 身份识别协议

步骤(1)~(3)等同于 SKID2，附加了以下两步：

- (1) Alice 向 Bob 发送消息： $H_K(R_B, A)$ 。其中， A 是 Alice 的姓名识别符。
- (2) Bob 计算 $H_K(R_B, A)$ ，并与收到的来自 Alice 的值进行比较。如果相等，那么他知道他正在与 Alice 进行通信。

对于中间人攻击来说，这一协议并不安全。一般来说，中间人攻击能够攻破不涉及某种秘密的任何协议。

7. 消息认证

当 Bob 收到来自 Alice 的消息时，他如何来判断这条消息是真的？如果 Alice 对这条消息进行数字签名，那么事情就变得十分容易了。Alice 的数字签名足以提示任何人她签发的这条消息是真的。

单钥密码体制也可以提供某种认证。当 Bob 收到某条采用共享密钥加密的消息时，他便知道此条消息来自 Alice。然而，Bob 却不能向 Trent 证明这条消息来自 Alice。Trent 只能知道这条消息来自 Bob 或者来自 Alice（因为没有其他任何人知道他们的共享密钥），但分不清这条消息究竟是谁发出的。

如果不采用加密，Alice 也可以采用消息认证码 MAC 的方法。采用这种方法也可以提示 Bob 有关消息的真伪，但它存在着与采用单钥加密体制相同的问题。

8.2.3 认证的密钥建立协议

这类协议将认证与密钥建立结合在一起，用于解决计算机网络中普遍存在的这样一个问题：Alice 和 Bob 是网络的两个用户，他们想通过网络进行安全通信。那么 Alice 和 Bob 如何才能做到在进行密钥交换的同时，确信她或他正在与另一方而不是 Mallory 通信呢？单纯的密钥建立协议有时还不足以保证在通信双方之间安全地建立密钥，与认证

相结合能可靠地确认双方的身份，实现安全密钥建立，使参与双方（或多方）确信没有其他人可以共享该秘密。密钥认证分为 3 种：

（1）隐式（Implicit）密钥认证：若参与者确信可能与他共享一个密钥的参与者的身份时，第二个参与者无须采取任何行动。

（2）密钥确证（Key Confirmation）：一个参与者确信第二个可能未经识别参与者，确实具有某个特定密钥。

（3）显式（Explicit）密钥认证：已经识别的参与者具有给定密钥。它具有隐式和密钥确证双重特征。

密钥认证的中心问题是识别第二参与者，而不是识别密钥值；而密钥确证则恰好相反，是对密钥值的认证。密钥确证通常包含了从第二参与者送来的消息，其中含有证据稍后可证明密钥的主权人。事实上密钥的主权人可以通过多种方式来证明，如生成密钥本身的一个单向杂凑值、采用密钥控制的杂凑函数以及采用密钥加密一个已知量等。这些技术可能会泄漏一些有关密钥本身的信息，而用零知识证明技术可以证明密钥的主权人但不会泄漏有关密钥的任何信息。

并非所有协议都要求实体认证，有些密钥建立协议（如非认证的 Diffie-Hellman 密钥协商协议）就不含实体的认证、密钥认证和密钥确证。单边（unilateral）密钥确证可能经常附有用最后消息推导密钥的单向函数。

在认证的密钥建立中，有基于身份的密钥建立协议，参与者的公钥中包含了身份信息（如名字、地址、身份号等），用来作为确定建立密钥的函数的输入变量。目前，许多协议都假设 Trent 与协议的参与者之间共享一密钥，并且所有这些密钥在协议开始执行前就已经分发到位。下面就来讨论这些协议，协议中采用的符号如表 8-11 所示。

表 8-11 认证和密钥交换协议中采用的符号

A	Alice 的姓名识别符	K	随机会话密钥
B	Bob 的姓名识别符	L	有效期
E_A	采用 Trent 与 Alice 共享的密钥加密	T_A, T_B	时戳
E_B	采用 Trent 与 Bob 共享的密钥加密	R_A, R_B	由 Alice 和 Bob 选择的一次随机数(Nonce)
I	索引号码	S_T	Trent 的签名

1. 大嘴青蛙协议

大嘴青蛙协议[Burrows 等 1989]可能是采用可信赖服务器的最简单的对称密钥管理协议。Alice 和 Bob 均与 Trent 共享一个密钥。此密钥只用做密钥分配，而不用来对用户之间传递的消息进行加密。只传送两条消息，Alice 就可将一个会话密钥发送给 Bob。协议描述如表 8-12 所示。

表 8-12 大嘴青蛙协议

大嘴青蛙协议

* 前提:

Alice 和 Bob 均与 Trent 共享一个密钥。此密钥只用做密钥分配，而不用来对用户之间传递的消息进行加密。

* 描述:

- (1) Alice 将时戳、Bob 的姓名以及随机会话密钥连接，并采用与 Trent 共享的密钥对整条消息加密。此后，将加密的消息和她的姓名一起发送给 Trent: $A, E_A(T_A, B, K)$ 。
- (2) Trent 对 Alice 发来的消息解密。之后，他将一个新的时戳、Alice 的姓名及随机会话密钥连接，并采用与 Bob 共享的密钥对整条消息加密。此后，将加密的消息发送给 Bob: $E_B(T_B, A, K)$

这个协议所做的一个最重要的假设是：Alice 完全有能力产生好的会话密钥。在实际中，真正随机数的生成是十分困难的。这个假设对 Alice 提出了很高的要求。

2. Yahalom 协议

在这一协议中，Alice 和 Bob 均与 Trent 共享一个密钥[Burrows 等 1989]。协议如表 8-13 所示。

表 8-13 Yahalom 协议

Yahalom 协议

* 前提: Alice 和 Bob 均与 Trent 共享一个密钥。

* 目标: Alice 和 Bob 均确信各自都在与对方进行对话，而不是与另外的第三方通话。

* 描述:

- (1) Alice 将其姓名和一个随机数链接在一起，发送给 Bob: A, R_A 。
- (2) Bob 将 Alice 的姓名、Alice 的随机数和他自己的随机数连接起来。并采用与 Trent 共享的密钥加密。此后将加密的消息和他的姓名一起发送给 Trent: $B, E_B(A, R_A, R_B)$ 。
- (3) Trent 生成两条消息。他先将 Bob 的姓名、某个随机的会话密钥、Alice 的随机数和 Bob 的随机数组合在一起，并采用与 Alice 共享的密钥对整条消息加密；其次将 Alice 的姓名和随机的会话密钥组合起来，并采用与 Bob 共享的密钥加密。最后将两条消息发送给 Alice: $E_A(B, K, R_A, R_B), E_B(A, K)$ 。
- (4) Alice 对第一条消息解密，提取出 K ，并证实 R_A 与在 (1) 中值相等。之后，Alice 向 Bob 发送两条消息。第一条消息来自 Trent，采用 Bob 的密钥加密；第二条是 R_B ，采用会话密钥 K 加密: $E_B(A, K), E_K(R_B)$ 。
- (5) Bob 用他的共享密钥对第一条消息解密，提取出 K ；再用该会话密钥对第二条消息解密求出 R_B ，并验证 R_B 是否与 (2) 中的值相同。

这个协议的新思路是：Bob 首先与 Trent 接触，而 Trent 仅向 Alice 发送一条消息。

3. Needham-Schroeder 协议

这个协议是由 R. Needham 和 M. Schroeder 设计的[Needham 等 1978]，协议采用了单钥体制和 Trent，无时戳，如表 8-14 所示。

表 8-14 Needham-Schroeder 协议

Needham-Schroeder 协议

- (1) Alice 向 Trent 发送一条消息: A, B, R_A 。其中包括她的姓名 A 、Bob 的姓名 B 和某个随机数 R_A 。
- (2) Trent 生成一个随机会话密钥 K 。他将会话密钥和 Alice 的姓名连接在一起, 并采用与 Bob 共享的密钥对其加密得到 $E_B(K, A)$; 此后, 他将 Alice 的随机数 R_A 、Bob 的姓名 B 、会话密钥 K , 以及上述加密的消息连接, 并采用与 Alice 共享的密钥加密。最后将加密的消息发送给 Alice: $E_A(R_A, B, K, E_B(K, A))$ 。
- (3) Alice 对消息解密求出 K , 并验证 R_A 就是她在(1)中发送给 Trent 的值。之后, 她向 Bob 发送消息: $E_B(K, A)$ 。
- (4) Bob 对收到的消息解密求出 K 。之后, 他生成另一随机数 R_B , 采用 K 加密后发送给 Alice: $E_K(R_B)$ 。
- (5) Alice 用 K 对收到的消息解密得到 R_B 。她生成 R_B-1 , 并采用 K 加密。最后, 将消息发送给 Bob: $E_K(R_B-1)$ 。
- (6) Bob 采用 K 对消息解密, 并验证得到的明文就是 R_B-1 。

这里采用 R_A , R_B 和 R_B-1 的目的是为了抗击重放攻击 (Replay Attack)。在实施攻击时, Mallory 可以记录前次执行协议时的一些旧消息, 然后重新发送它们, 试图攻破协议。在 (2) 中, R_A 的出现使 Alice 确信: Trent 的消息是合法的, 并非是重发上次协议执行中的旧消息。当 Alice 成功解密, 求出 R_B , 并在 (5) 中向 Bob 发送 R_B-1 时, Bob 确信 Alice 的消息是合法的, 而不是重发上次协议执行中的旧消息。

这一协议的主要安全漏洞是旧会话密钥存在着脆弱性。如果 Mallory 能够获得某个旧的会话密钥, 他就可以成功地对协议发起攻击[Denning 1982]。他要做的就是记录 Alice 在 (3) 中发给 Bob 的消息。之后, 一旦得到 K , 他就可以假装成 Alice 对协议发起攻击:

- (1) ~ (2) 与 Needham-Schroeder 协议相同。
- (3) Mallory 假装成 Alice 向 Bob 发送消息: $E_B(K, A)$ 。
- (4) Bob 解密求出 K , 生成 R_B , 并发送给 Alice 消息: $E_K(R_B)$ 。
- (5) Mallory 截获这一消息, 并用 K 对其解密。此后, 将发给 Bob: $E_K(R_B-1)$ 。
- (6) Bob 验证“Alice”的消息是 R_B-1 。

至此, Mallory 已使 Bob 相信他正在与 Alice 通话。

在协议中采用时戳, 可以提高协议的安全性, 从而有效地抗击这种攻击[Denning 等 1981, Denning 1982]。在 (2) 中, 时戳被加入到 Trent 发送的消息中, 即 $E(K, A, T)$ 。时戳要求系统有一个安全的和精确的时钟, 然而要做到这点并非易事。

如果 Trent 与 Alice 共享的密钥被泄漏, 那么后果更加严重。Mallory 可以用它获得会话密钥, 然后与 Bob (或其他任何想要与之对话的用户) 进行通信。更糟的是, 在 Alice 改变了她的密钥后, Mallory 还可以继续进行这种攻击[Bauer 等 1983]。

为了克服原协议存在的问题, Needham 和 Schroeder 对上述协议做了改进, 提出了一种安全性更高的协议[Needham 等 1987]。此新协议与将要讨论的 Otway-Rees 协议基本相同。

4. Otway-Rees 协议

这一协议也采用了单钥密码体制。该协议有 Trent 参与, 无时戳[Otway 等 1987]。协议描述如表 8-15 所示。

表 8-15 Otway-Rees 协议

Otway-Rees 协议

★ 目标：Alice 和 Bob 相互确认对方的身份，并获得一个通信用的密钥。

★ 描述：

- (1) Alice 生成一条消息，其中包括索引号码、她的姓名、Bob 的姓名和一个随机数，并将这条消息采用她与 Trent 共享的密钥加密。此后，将密文连同索引号、Alice 和 Bob 的姓名一起发送给 Bob: $I, A, B, E_A(R_A, I, A, B)$ 。
- (2) Bob 生成一条消息，其中包括一个新的随机数、索引号、Alice 和 Bob 的姓名，并采用他与 Trent 共享的密钥对这条消息加密。此后，将密文连同 Alice 的密文、索引号、Alice 和 Bob 的姓名一起发送给 Trent: $I, A, B, E_A(R_A, I, A, B), E_B(R_B, I, A, B)$ 。
- (3) Trent 生成一个随机的会话密钥。此后，生成两条消息。第一条消息采用他与 Alice 共享的密钥对 Alice 的随机数和会话密钥加密；第二条采用他与 Bob 的共享密钥对 Bob 的随机数和会话密钥加密。最后 Trent 将这两条消息连同索引号一起发送给 Bob: $I, E_A(R_A, K), E_B(R_B, K)$ 。
- (4) Bob 将属于 Alice 的那条消息连同索引号一起发送给 Alice: $I, E_A(R_A, K)$ 。
- (5) Alice 对收到的消息解密得到随机数 R_A 和会话密钥。如果 R_A 与 (1) 中的值相同，那么 Alice 确认随机数和会话密钥没有被改动过，并且不是重发某个旧会话密钥。

5. Kerberos 协议

Kerberos 协议是从 Needham-Schroeder 协议演变而来的。有关该协议的详细介绍，请参阅 15.6 节。在基本的 Kerberos V.5 协议中，Alice 和 Bob 各自与 Trent 共享一个密钥，采用时戳，Alice 与 Bob 通信的会话密钥由 Alice 生成。

Kerberos 协议描述如表 8-16 所示。

表 8-16 Kerberos 协议

Kerberos 协议

★ 前提：

每个用户均具有一个与 Trent 同步的时钟。

★ 描述：

- (1) Alice 向 Trent 发送她的身份和 Bob 的身份: A, B 。
- (2) Trent 生成一条消息，其中包含时戳、有效期 L 、随机会话密钥和 Alice 的身份，并采用与 Bob 共享的密钥加密。此后，他将时戳、有效期、会话密钥和 Bob 的身份采用与 Alice 共享的密钥加密。最后，将这两条加密的消息发送给 Alice: $E_A(T, L, K, B), E_B(T, L, K, A)$ 。
- (3) Alice 采用 K 对其身份和时戳加密，并连同从 Trent 收到的、属于 Bob 的那条消息发送给 Bob: $E_K(A, T), E_B(T, L, K, A)$ 。
- (4) Bob 将时戳加 1，并采用 K 对其加密后发送给 Alice: $E_K(T+1)$ 。

实际上，同步时钟是由系统中的安全时间服务器来保持的。通过设立一定的时间间隔，系统可以有效地检测到重放攻击。

6. Neuman-Stubblebine 协议

无论是系统故障还是计时误差，都有可能使时钟失步。若发生时钟失步，所有依赖于同步时钟的协议都有可能遭到攻击[Gong 1992]。如果发送者的时钟超前于接收者的时钟，Mallory 可以截获发送者的某个消息，等该消息中的时戳接近于接收者的时钟时再重

发这条消息。此攻击被称做等待重放攻击 (Suppress-Replay Attack)，它造成的后果是十分严重的。

Neuman-Stubblebine 协议首先在文献[Kehne 等 1992]中提出，此后又在文献[Neuman 等 1993]中进行了改进。它的特点是能够抵抗等待重放攻击。作为 Yahalom 协议的加强版本，Neuman-Stubblebine 协议抵抗是一个很好的协议。该协议的描述如表 8-17 所示。

表 8-17 Neuman-Stubblebine 协议

Neuman-Stubblebine 协议

* 目标: Alice 和 Bob 相互确认对方的身份，并共享一个会话密钥。

* 描述:

- (1) Alice 将她的姓名和某个随机数连接起来，发送给 Bob: A, R_A 。
- (2) Bob 将 Alice 的姓名、随机数和时戳连接起来，并采用与 Trent 共享的密钥加密。此后，将密文连同他的姓名、新产生的随机数一起发送给 Trent: $B, R_B, E_B(A, R_A, T_B)$ 。
- (3) Trent 生成一随机的会话密钥。之后，他生成两条消息：第一条是采用与 Alice 共享的密钥对 Bob 的身份、Alice 的随机数、会话密钥和时戳加密；第二条是采用与 Bob 共享的密钥对 Alice 的身份、会话密钥和时戳加密。最后，他将这两条消息连同 Bob 的随机数一起发送给 Alice: $E_A(B, R_A, K, T_B), E_B(A, K, T_B), R_B$ 。
- (4) Alice 对属于她的消息解密得到会话密钥 K ，并确认 R_A 与在 (1) 中的值相等。此后，Alice 发送给 Bob 两条消息：第一条消息来自 Trent，第二条消息是采用会话密钥对 R_B 加密: $E_B(A, K, T_B), E_K(R_B)$ 。
- (5) Bob 对第一条消息解密得到会话密钥 K ，并确认 T_B 和 R_B 的值与 (2) 中的值相同。

这个协议不需要同步时钟，因为时戳仅与 Bob 的时钟有关，Bob 只对他自己生成的时戳进行检查。

这个协议的优点是：在预定的时限内，Alice 能够将收自 Trent 的消息用于随后与 Bob 的认证中。假设 Alice 和 Bob 已经完成了上述协议，并建立连接开始通信，但由于某种原因连接被中断。在这种情况下，Alice 和 Bob 不需要 Trent 的参与，仅执行 3 步就可以实现相互认证。此时，协议的执行过程如下：

- (1) Alice 将 Trent 在 (3) 中发给她的消息，连同一个新随机数一起发送给 Bob: $E_B(A, K, T_B), R'_A$ 。
- (2) Bob 采用会话密钥对 Alice 的随机数加密，连同一个新的随机数发送给 Alice: $R'_B, E_K(R'_A)$ 。
- (3) Alice 采用会话密钥对 Bob 的新随机数加密，并发送给 Bob: $E_K(R'_B)$ 。

在上述协议中，采用新随机数的目的是为了防止重放攻击。

7. DASS

分布认证安全服务 (Distributed Authentication Security Service, DASS) 协议是由 DEC (Digital Equipment Corporation) 公司开发的，其目的也是为了提供双向认证和密钥交换。与前面介绍的协议不同，DASS 既采用了双钥密码体制，也采用了单钥密码体制。该协议假设 Alice 和 Bob 各自具有一个私钥，而 Trent 掌握着他们的签名公钥。协议的描述如表 8-18 所示。

表 8-18 DASS 协议

DASS 协议

- (1) Alice 将 Bob 的身份发送给 Trent: B 。
- (2) Trent 将 Bob 的公钥和身份连接, 并采用其私钥 T 对消息进行数字签名: $S_T(B, K_B)$, 发送给 Alice。
- (3) Alice 对 Trent 的签名加以验证, 以证实她收到的公钥就是 Bob 的公钥。她生成一个会话密钥和一个随机的公钥/私钥对 K_p , 并用 K 对时戳加密。接下来, 她采用私钥 K_A 对会话密钥的有效期 L 、自己的身份和 K_p 进行签名。之后, 她采用 Bob 的公钥对会话密钥 K 加密, 再用 K_p 对其签名。最后, 她将所有的消息发送给 Bob: $E_K(T_A), S_K(L, A, K_p), S_{K_p}(E_{K_p}(K))$ 。
- (4) Bob 将 Alice 的身份发送给 Trent (这里的 Trent 可以是另外一个实体): A 。
- (5) Trent 将 Alice 的公钥和身份连接, 并采用其私钥 T 对消息进行数字签名: $S_T(A, K_A)$, 发送给 Bob。
- (6) Bob 验证 Trent 的签名, 以证实他收到的公钥就是 Alice 的公钥。此后, 他验证 Alice 的签名并得到 K_p 。他再采用 K_p 验证 $S_{K_p}(E_{K_p}(K))$, 并采用他的私钥解密得到会话密钥 K 。最后, 他采用 K 对 $E_K(T_A)$ 解密得到时戳 T_A , 确认这条消息是当前发送的, 而不是重发某条旧消息。
- (7) 如果需要进行相互认证, Bob 采用 K 对一个新时戳加密后发送给 Alice: $E_K(T_B)$ 。
- (8) Alice 采用 K 对收到的消息解密, 并确认此消息是当前发送的, 而不是重发过去某条消息。

基于 DASS, DEC 公司又开发出新的协议 SPX。此协议的详细情况请参阅文献 [Alagappan 等 1991]。

8. Denning-Sacco 协议

这个协议也采用了双钥体制[Denning 等 1981]。此协议假设 Trent 掌握了所有用户的公钥数据库。协议描述如表 8-19 所示。

表 8-19 Denning-Sacco 协议

Denning-Sacco 协议

- (1) Alice 向 Trent 发送她的身份和 Bob 的身份: A, B 。
- (2) Trent 采用其私钥对 Bob 的公钥和 Alice 的公钥签名, 并发送给 Alice: $S_T(B, K_B), S_T(A, K_A)$ 。
- (3) Alice 首先采用其私钥对一个随机的会话密钥和时戳签名, 再采用 Bob 的公钥加密。最后, 将结果连同收到的两个签名公钥一起发送给 Bob: $E_B(S_A(K, T_A)), S_T(B, K_B), S_T(A, K_A)$ 。
- (4) Bob 采用其私钥对收到的消息解密, 此后采用 Alice 的公钥对 Alice 的签名进行验证。最后, 检验时戳是否仍然有效。

至此, Alice 和 Bob 都具有一个会话密钥, 他们可以它进行安全的通信。

Denning-Sacco 协议看似安全, 其实不然。在 Bob 与 Alice 一起完成协议后, Bob 可以假冒成 Alice。通过表 8-20 会看到 Bob 是如何假冒 Alice 的。

这个问题很容易得到解决。只要将网络用户的身份加入到 (3) 中的加密消息中, 就可以成功地防止这种假冒攻击: $E_B(S_A(A, B, K, T_A)), S_T(B, K_B), S_T(A, K_A)$ 。

现在, Bob 就无法将旧的消息重发给 Carol, 因为在数字签名项中已经清楚地表明通信是在 Alice 和 Bob 两个用户之间进行。

表 8-20 假冒攻击

假冒攻击

- (1) Bob 将他的身份和 Carol 的身份发送给 Trent: B, C 。
- (2) Trent 将 Bob 和 Carol 的签名公钥发送给 Bob: $S_T(B, K_B), S_T(C, K_C)$ 。
- (3) Bob 将过去收自 Alice 的签名会话密钥和时戳, 采用 Carol 的公钥进行加密, 并连同 Alice 和 Carol 的公钥证明 (Certificate) 一起发送给 Carol: $E_C(S_A(K, T_A)), S_T(A, K_A), S_T(C, K_C)$ 。
- (4) Carol 采用其私钥对收到的消息 $E_C(S_A(K, T_A))$ 解密, 然后采用 Alice 的公钥对签名加以验证。最后, 检查时戳是否仍然有效。

至此, Carol 认为他正在与 Alice 进行通信, Bob 已成功地假冒成 Alice。实际上, 在时戳的有效期内, Bob 可以假冒网上的任何用户。

9. Woo-Lam 协议

这个协议也采用了双钥体制[Woo 等 1992]。协议的描述如表 8-21 所示。

表 8-21 Woo-Lam 协议

Woo-Lam 协议

- (1) Alice 向 Trent 发送她的身份和 Bob 的身份: A, B 。
- (2) Trent 采用其私钥 T 对 Bob 的公钥 K_B 进行签名并发送给 Alice: $S_T(K_B)$ 。
- (3) Alice 验证 Trent 的签名。此后, 采用 Bob 的公钥对她的身份和产生的随机数加密, 并发送给 Bob: $E_{K_B}(A, R_A)$ 。
- (4) Bob 采用 Trent 的公钥 K_T 对 Alice 的随机数加密, 并连同对他的身份、Alice 的身份一起发送给 Trent: $A, B, E_{K_T}(R_A)$ 。
- (5) Trent 用其私钥对 Alice 的公钥 K_A 进行签名后发送给 Bob。同时, 他也对 Alice 的随机数、随机会话密钥、Alice 的身份、Bob 的身份进行签名, 再用 Bob 的公钥加密后发送给 Bob: $S_T(K_A), E_{K_B}(S_T(R_A, K, A, B))$ 。
- (6) Bob 验证 Trent 的签名。此后, 他对 (5) 中消息的第二部分解密, 并再采用 Alice 的公钥对得到的 Trent 的签名值和一个新随机数 R_B 加密, 将结果发送给 Alice: $E_{K_A}(S_T(R_A, K, A, B), R_B)$ 。
- (7) Alice 验证 Trent 的签名和她的随机数 R_A 。此后, 她采用会话密钥 K 对 Bob 的随机数 R_B 加密后, 发送给 Bob: $E_K(R_B)$ 。
- (8) Bob 对收到的消息解密得到随机数 R_B , 并检查它是否被改动过。

10. EKE 协议

加密密钥交换 (Encrypted Key Exchange, EKE) 协议是由 S. Bellovin 和 M. Merritt [Bellovin 等 1992] 提出的。协议既采用了单钥体制, 也采用了双钥体制。它的目的是为计算机网络上的用户提供安全性和认证业务。这个协议的新颖之处是: 采用共享密钥来加密随机生成的公钥。通过运行这个协议, 两个用户可以实现相互认证, 并共享一个会话密钥 K 。

协议假设 Alice 和 Bob (他们可以是两个用户, 也可以是一个用户、一个主机) 共享一个口令 P 。协议描述如表 8-22 所示。

表 8-22 EKE 协议

EKE 协议

- (1) Alice 生成一随机的公钥/私钥对。她采用单钥算法和密钥 P 对公钥 K' 加密,并向 Bob 发送以下消息: $A, E_P(K')$ 。
- (2) Bob 采用 P 对收到的消息解密得到 K' 。此后,他生成一个随机会话密钥 K ,并采用 K' 对其加密,再采用 P 加密,最后将结果发送给 Alice: $E_P(E_{K'}(K))$ 。
- (3) Alice 对收到的消息解密得到 K 。此后,她生成一个随机数 R_A ,用 K 加密后发送给 Bob: $E_K(R_A)$ 。
- (4) Bob 对消息解密得到 R_A 。他生成另一个随机数 R_B ,采用 K 对这两个随机数加密后发送给 Alice: $E_K(R_A, R_B)$ 。
- (5) Alice 对消息解密得到 R_A, R_B 。假设收自 Bob 的 R_A 与 (3) 中发送的值相同, Alice 便采用 K 对 R_B 加密,并发送给 Bob: $E_K(R_B)$ 。
- (6) Bob 对消息解密得到 R_B 。假设收自 Alice 的 R_B 与在 (4) 中 Bob 发送的值相同,协议就完成了。通信双方可以采用 K 作为会话密钥。

EKE 可以采用各种双钥算法来实现,例如 RSA、ElGamal 和 Diffie-Hellman 协议等。

选用和设计何种类型的协议要根据实际应用对确认的要求以及实现的机制来定,需要考虑多方面的因素,主要有:

- (1) 认证的特性,是实体认证、密钥认证和密钥确认的任何一种组合。
- (2) 认证的互易性 (Reciprocity),认证可能是单方的,也可能是相互的。
- (3) 密钥的新鲜性 (Freshness),保证所建立的密钥是新的。
- (4) 密钥的控制,有的协议由一方选定密钥值,有的则通过协商由双方提供的信息导出,不希望由单方来控制或预先定出密钥值。
- (5) 有效性,包括参与者之间交换的消息次数、传送的数据量、各方计算的复杂度,以及减少实时在线计算量的可能性等。
- (6) 第三方参与,包括有关第三方参与,在有第三方参与时是联机还是脱机参与,以及对第三方信赖程度。
- (7) 是否采用证书,以及证书的类型。
- (8) 不可否认性,可能提出收据证明已收到交换的密钥。

有关密钥建立协议的研究,可参阅[Desmedt 1988, 1994; Diffie 等 1992; Maurer 1993]。

8.3**秘密分拆协议**

假设你发明了一种饮料,但又不想让竞争对手知道该饮料的配方,那么就必须对饮料中所含的各种成分的比例加以保密。在生产过程中,你可能将配方告诉最信赖的几个雇员。但是,如果他们中的一个背叛了你而跑到竞争对手一边时,秘密就会完全泄漏。

不久，你的对手就可能生产出和你完全一样的产品。

在现实中，如何解决这类问题呢？这就涉及秘密分拆的问题。人们往往将某条消息分成许多碎片[Feistel 1970]，从每一碎片本身不会看出什么东西。但是，如果将所有的碎片重新组合在一起，就会重显消息。拿上面的例子来说，如果每个雇员只掌握配方中一种成分的比例，那么只有所有的雇员在一起才能够生产出这种饮料。任何一个雇员的离开只能带走属于他的那部分秘密，这部分秘密将毫无用处。

最简单的秘密分拆方案是将某条消息分给两个人。下面介绍一种秘密分拆协议，这里 Trent 将某条消息分给 Alice 和 Bob，如表 8-23 所示。

表 8-23 秘密分拆协议

秘密分拆协议
(1) Trent 生成一个随机比特串 R ，它与消息 M 具有相同的长度。
(2) Trent 将 M 和 R 进行异或运算，得到 S ： $S = M \oplus R$ 。
(3) Trent 将 R 分给 Alice，将 S 分给 Bob。 若想重组这条消息，Alice 和 Bob 仅需执行下一步。
(4) Alice 和 Bob 将各自得到的比特串进行异或运算，就会得到消息： $S = R \oplus S$ 。

这一技术是绝对安全的。每个消息碎片本身毫无价值。从本质上看，Trent 采用一次随机数对消息加密，此后将密文分发给一个人，而将一次随机数又分发给另外一个人。在前面已经讨论过一次一密体制，它具有绝对的安全性。无论计算能力有多高，均不会从某一碎片中推出消息本身。

这一方案很容易推广到有多个人的情况。要将一条消息分拆成多份，就要采用多个随机数对消息进行异或运算。在如表 8-24 所示的例子中，Trent 将消息分成了 4 份。

表 8-24 裁决协议

裁决协议
(1) Trent 生成 3 个随机比特串 R 、 S 和 T ，它们与消息 M 具有相同的长度。
(2) Trent 将 3 个比特串与消息 M 异或，得到 U ： $U = M \oplus R \oplus S \oplus T$ 。
(3) Trent 将 R 发送给 Alice， S 发送给 Bob， T 发送给 Carol， U 发送给 Dave。 Alice、Bob、Carol 和 Dave 这 4 个人在一起，就可以重组这条消息。
(4) Alice、Bob、Carol 和 Dave 集合在一起，计算： $M = R \oplus S \oplus T \oplus U$ 。

在这一协议中，Trent 有绝对的权力，并且可以做他想做的任何事情。他可以把毫无意义的东西拿出来，并声称这是消息的一个有效组成部分。在重组这条秘密消息之前，没有人知道这件事。他可以将分拆的消息碎片分发给 Alice、Bob、Carol 和 Dave。在解雇 Bob 时，他会告诉每个人只有 Alice、Carol 和 Dave 掌握的消息碎片可以重组消息，而 Bob 的那份消息碎片毫无用处。因为这条秘密的消息是由 Trent 来分割的，所以 Trent 知道这条秘密的消息。

然而，这个协议存在一个问题：如果任何一部分消息碎片丢失了，并且 Trent 不在

现场，那么其他人无法重组这一消息，这等于丢失了这条消息。如果 Carol 知道饮料的一部分配方，并将其带走为对手工作，那么其他人就会陷入困境。虽然 Carol 不能采用他带走的那部分秘密生产出相同的饮料，但是对 Alice、Bob 和 Dave 来说也一样。由于 R 、 S 、 T 和 U 的长度与 M 相同，他们除了知道消息的长度以外，对其他一无所知。在 8.4 节中，将讨论如何解决这个问题。

8.4

会议密钥分配和秘密广播协议

8.4.1 秘密广播协议

Alice 想通过一个发射机广播一条消息 M ，然而她不打算让所有的听众都能听懂。她仅想有选择地让部分听众听懂她的消息 M ，而其他人什么也听不到。

第一种方法：Alice 可以与每个听众共享一个不同的密钥（秘密的或公开的）。她用某个随机密钥 K 对消息 M 加密，然后用预定接收者的密钥对 K 加密（记为 K_s ）。最后，她将加密的消息和所有加密的密钥 K_s 广播出去。收听者 Bob 采用他的密钥对所有 K_s 解密，并寻找那个正确的密钥 K ，再用它对消息解密得到 M ；若 Alice 不介意让人知道她发送的消息是给谁的，那么她可以在 K_s 的后面附加上预定接收者的姓名。接收者只需搜索各自的姓名，并对相应的 K_s 解密即可。

第二种方法：这一方法在文献[Chiou 等 1989]中做了介绍。首先，每个听众与 Alice 共享一个密钥 K_s ，这个密钥比所有加了密的消息都大。所有这些密钥都是两两互素的。Alice 采用某个随机密钥 K 对消息加密。此后，她生成一个整数 R ，使得当某个密钥要用来对消息解密时， $R \equiv K \pmod{K_s}$ ；否则 $R \equiv 0 \pmod{K_s}$ 。

例如，若 Alice 想要 Bob、Carol 和 Ellen 接收到她发送的消息，而不让 Dave 和 Frank 接收到，那么她用 K 对消息加密，继而计算 R ，使得：

$$R \equiv K \pmod{K_B}$$

$$R \equiv K \pmod{K_C}$$

$$R \equiv 0 \pmod{K_D}$$

$$R \equiv K \pmod{K_E}$$

$$R \equiv 0 \pmod{K_F}$$

这是一个纯代数问题，Alice 很容易求出 R 。当听众收到这一广播时，他们各自对接收到的密钥取模 K_s 。如果他们被允许接收消息，他们就能够恢复出密钥，否则，他们什么也不会得到。

第三种方法：文献[Berkovitz 1991]提出了一种采用了门限方案的方法。像其他方法一样，每个可能的接收者都可以得到一个密钥，这个密钥是尚未建立的门限方案的“投影”。Alice 也为自己准备一些密钥，给系统增加某些随机性。

首先，假设有 k 个听众。在广播消息 M 时，Alice 用密钥 K 对消息 M 加密，并进行以下操作：

(1) Alice 选择一个随机数 j 。这个随机数用于隐藏消息接收者的数目。这个数不必很大，它可以是一个很小的数。

(2) Alice 建立一个 $(k+j+1, 2k+j+1)$ 门限方案，其中： K 是密钥；预定接收者的密钥就是这一门限方案的“投影”；非预定接收者的密钥不是“投影”； j 是随机选择的“投影”个数，它们与任何一个密钥均不同。

(3) Alice 广播 $k+j$ 个随机选择的“投影”，其中任何一个都不是 (2) 中列出的“投影”。

(4) 所有收到这一广播的听众将他们各自的“投影”加到所接收的 $k+j$ 个“投影”上。如果加上该“投影”后能够计算出密钥 K ，那么他们就恢复出密钥，从而就可解密出用 K 加密的消息 M ；如果加上该“投影”后不能够计算出密钥 K ，那么他们就不能恢复出密钥，从而就不能解密出用 K 加密的消息 M 。

此外，文献[Gong 1994]还介绍了其他方法。

8.4.2 会议密钥分配协议

这个协议将实现一组 (n 个) 用户通过不安全的信道共享某个密钥。这一组用户共享两个大素数 p 和 q ，生成元 g 与 q 具有相同的长度。协议如表 8-25 所示。

表 8-25 会议密钥分配协议

会议密钥分配协议

(1) 用户 $i(i=1,2,\dots,n)$ ，选择随机数 $r_i < q$ ，并广播： $z_i = g^{r_i} \bmod p$ 。

(2) 每个用户验证： $z_i^q \equiv 1 \pmod p$ ， $i=1,2,\dots,n$ 。

(3) 用户广播： $x_i = (z_{i+1} / z_{i-1})^{r_i} \bmod p$ 。

(4) 用户计算： $K = (z_{i-1})^{nr_i} \times x_i^{n-1} \times x_{i+1}^{n-2} \times \dots \times x_{i-2}^{x+1} \bmod p$ 。

在上面的协议中，所有下标 $i-1, i-2$ 和 $i+1$ 的计算都是模 n 运算。在协议执行完后，所有组内用户均共享相同的密钥 K ，组外人均得不到任何有用信息。

这个协议的缺点是不能抵抗中间人攻击。在文献[Ingemarsson 等 1982]中，作者提出了另外一种会议密钥分配协议。

8.5

密码协议的安全性

认证协议是许多分布系统安全的基础。确保这些协议能够安全地运行是极为重要的。虽然认证协议中仅仅进行很少的几组消息传输，但是其中的每一消息的组成都是经过巧妙设计的，而且这些消息之间有着复杂的相互作用和制约。在设计认证协议时，人们通常采用不同的密码体制。而且所设计的协议也常常应用于许多不同的通信环境。但是，现有的许多协议在设计上普遍存在着某些安全缺陷。造成认证协议存在安全漏洞的原因有很多，但主要的原因有如下两个：①协议设计者有可能误解了所采用的技术，或者不

恰当地照搬了已有协议的某些特性；②人们对某一特定的通信环境及其安全需求研究不够。人们很少知道所设计的协议是如何才能够满足安全需求的。因此，在近来出现的许多协议中都发现了不同程度的安全缺陷或冗余消息。

本节将讨论对协议的攻击方法和安全性分析方法。

8.5.1 对协议的攻击

在分析协议的安全性时，常用的方法是对协议施加各种可能的攻击来测试其安全度。密码攻击的目标通常有3个：第一是协议中采用的密码算法；第二是算法和协议中采用的密码技术；第三是协议本身。由于本节仅讨论密码协议，因此，只考虑对协议自身的攻击，而假设协议中所采用的密码算法和密码技术均是安全的。对协议的攻击可以分为被动攻击和主动攻击。

被动攻击是指协议外部的实体对协议执行的部分或整个过程实施窃听。攻击者对协议的窃听并不影响协议的执行，他所能做的是对协议的消息流进行观察，并试图从中获得协议中涉及的各方的某些信息。他们收集协议各方之间传递的消息，并对其进行密码分析。这种攻击实际上属于一种惟密文攻击。被动攻击的特点是难以检测，因此在设计协议时应该尽量防止被动攻击，而不是检测它们。

主动攻击对密码协议来说具有更大的危险性。在这种攻击中，攻击者试图改变协议执行中的某些消息以达到获取信息、破坏系统或获得对资源的非授权访问。他们可能在协议中引入新的消息，删除消息，替换消息，重发旧消息，干扰信道或修改计算机中存储的信息。在网络环境下，当通信各方彼此互不信赖时，这种攻击对协议的威胁显得更为严重。攻击者不一定是局外人，他可能就是一个合法用户，可能是一个系统管理者，可能是几个人联手对协议发起攻击，也可能就是协议中的一方。

若主动攻击者是协议涉及的一方，则称其为欺骗者（Cheater）。他可能在协议执行中撒谎，或者根本不遵守协议。欺骗者也可以分为主动欺骗者和被动欺骗者。被动欺骗者遵守协议，但试图获得协议之外更多的信息；主动欺骗者则不遵守协议，对正在执行的协议进行干扰，试图冒充它方或欺骗对方，以达到各种非法目的。

如果协议的参与者中多数都是主动欺骗者，那么就很难保证协议的安全性。但是，在某些情况下，合法用户可能会检测到主动欺骗的存在。显然，密码协议对于被动欺骗应该是安全的。

在实际中，对协议的攻击方法是多种多样的。对不同类型的密码协议，存在着不同的攻击方法。很难将所有攻击方法一一列出，这里仅仅对几种常用的攻击方法进行详细介绍。为了便于理解，下面结合一个具体的协议对这些攻击方法加以说明。

图8-4（a）为一个单向用户认证协议，实现用户B对用户A的认证功能。其中， N 为一次随机数， $E_a(N)$ 表示采用密钥 K_a 对 N 加密。 K_a 要么是用户A与用户B的共享密钥，要么是用户A的公钥。图8-4（b）的协议与图8-4（a）完全对称，它实现用户A对用户B的认证。将图8-4（a）和图8-4（b）的两个协议结合起来，就得到图8-4（c）的

双向认证协议；将其做进一步的简化，便得到图 8-4 (d) 的协议。对于单钥体制来说， K_a 和 K_b 是相同的，因此 $E_a(N)$ 和 $E_b(N)$ 表示采用同一个共享密钥对随机数加密。

图 8-4 (d) 中的双向认证协议是由两个单向认证协议演化而来的。初看似乎无可挑剔，但是它却不安全。我们将证明，当此协议采用单钥体制构造时，攻击者很容易采用不同的攻击方法攻破此协议。在后面，我们将结合这个例子，讨论几个典型的攻击协议的方法。

1. 已知明文攻击

图 8-4 (d) 所示的协议的一个缺点是它对已知明文攻击的开放性。对于 A 和 B 之间交换的每个密文消息比特流，均可以在随后的消息流中找到相应的明文。在每次执行协议时，被动攻击者可以通过搭线窃听的方法，收集到两个明文-密文对。通过长期不断地窃听，攻击者至少可以建立起一个加密表，甚至可以根据所采用的加密算法强度，进一步攻破此方案并发现加密密钥。因此，在设计认证协议时，一般要求所交换的加密消息的相应明文不会被攻击者得到或推出。

2. 选择密文攻击

当攻击者将已知明文攻击转化为选择密文攻击时，他所起的作用是主动性的而不是被动性的，这种威胁就更为严重。在图 8-4 (d) 中，攻击者可以假扮成 A 或 B ，向另一方 (B 或 A) 发送某个经过选择的密文消息，并等待对方发送回相应的解密数值。攻击者并不知道确切的密钥是什么，当然就不会完成第 3 个消息流。然而，他可以积累关于明文-密文对的有关知识，其中的密文是经他自己精心选择的（或者当他发送的消息是明文时，收到的回执将是相应的密文）。他可以尝试采用特定的密文比特串，如全“0”、全“1”或其他消息，来更快地解出密钥。因此，在设计协议时，通常期望攻击者不能欺骗合法用户来获取选择密文的相应明文，或者选择明文的相应密文。

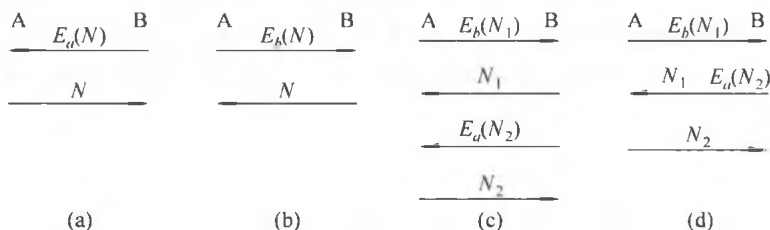


图 8-4 用户认证协议举例

3. 预言者会话攻击

实际上，在上述简单协议中，如果 A 和 B 采用相同的密钥，攻击者无须破译出密钥就能攻破此认证协议。这种攻击如图 8-5 所示。在图 8-5 中，攻击者 X 假装成 A ，通过向 B 发送某个加密的随机数 $E(N_1)$ 开始会话。 B 则响应此会话请求，向 X 发回解密的消息 N_1 和某个加密的随机数 $E(N_2)$ 。虽然 X 不能对 $E(N_2)$ 解密得到 N_2 ，但是他可以通过对 A 实施选择密文攻击来得到 N_2 。 A 作为预言者向 X 提供必要的解密值 N_2 。首先，攻

击者假装成用户 B ，通过向 A 发送加密消息 $E(N_2)$ 开始会话。 A 则响应会话请求发回 N_2 和某个加密的随机数 $E(N_3)$ 。一旦 X 获得了 N_2 ，就会抛弃 A ，转过来与 B 会话，而根本不去考虑如何解密 $E(N_3)$ 。通过向 B 发送 N_2 ，攻击者 X 就会成功地假冒用户 A ，取得用户 B 的信赖而完成认证协议。

这个例子暴露了协议中存在的一个基本缺陷，因此，在设计认证协议时，协议每个消息流中用到的密码消息必须有所区别，使得攻击者不可能从第 2 个消息流中推出、重组或伪造出第 3 个消息流中必需的消息。

实际上，认证协议的这个缺陷已经被发现，并得到了改进，成为 ISO SC27 标准协议。在这个协议中，协议的发起者 A 发送的“提问”消息是为了使对方表明其具有加密某个给定明文的能力，而 B 发出的“提问”是为了让对方表明其具有解密某个给定密文的能力。这样，攻击者便不会将一方用做预言者“解密服务器”去对付另一方。此时也许有人肯定地认为此协议不会有其他缺陷，但遗憾的是，改进后的协议仍然存在着缺陷。实际上，这个缺陷在原始的协议中同样存在。下面将对此加以分析。

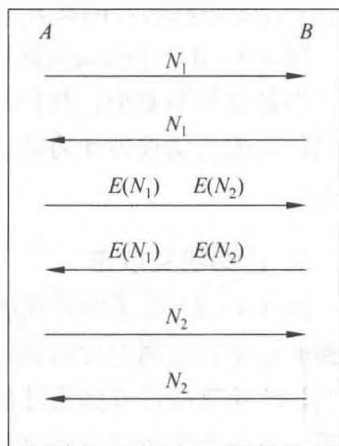


图 8-5 预言者会话攻击

4. 并行会话攻击

对于上面讨论的协议，我们发现它具有一个普遍性的缺陷，即不能抵抗并行会话攻击。这里，攻击者所起的作用是被动的，而非主动的。

首先，攻击者 X 截获由 A 向 B 发出的“提问”随机数 N_1 ，立刻反手将其发送给 A 。这里，攻击者根本不理睬 B ，而把 A 变成对付他自己的预言者。由于攻击者 X 不能对 A 的“提问” N_1 做出相应的“回答” $E(N_1)$ ，他只有假装成 B 试图与 A 进行会话。显然，攻击者 X 选择了 N_1 作为对 A 发出的“提问”，让 A 来替他精确地计算完成认证所必需的响应消息 $E(N_1)$ 。同时，在并行会话中， A 也发送出它自己的加密“提问”消息 $E(N_2)$ 。 X 在获得 $E(N_1)$ 和 $E(N_2)$ 后，立即将它们回送给 A 。 A 发送 N_2 完成第一次认证交换过程，而 N_2 又恰恰就是 X 为完成第二次认证交换所必需的数值。这样， X 便在原始会话及其并行会话中成功地扮演了 B 的角色。

对于不同的网络结构，在不同协议层上的许多连接建立协议往往不允许同时建立多个并行会话。然而，在某些现存的网络环境下，这种并行会话在设计上是允许的。在这些允许的网络环境下设计密码协议时，设计者必须小心对待这种攻击。协议必须能够检测在某次会话中收到的第一个“提问”不是重发另外某个会话中的“提问”。然而，将会话的安全性留给用户去考虑是十分危险的，必须在进行协议设计时就尽量避免这种攻击。

并行会话攻击揭示了许多简单认证协议存在的另一个基本的缺陷。要克服这种缺陷,协议第二个消息流中的密码表达式就必须是非对称的,也就是与方向有关,使得由 A 发起的协议中的值,不能用于由 B 发起的协议之中。

基于以上的考虑,即:在第二个消息流中的密码消息必须是非对称的(具有方向性),并且要与第 3 个消息流有所区别。也许有人提出如图 8-6 所示的协议,并认为它是安全的。在图中,对随机数 N_1 加密已换成对 N_1 的函数加密(xor 表示异或运算)。它除了仍然可以遭到已知明文或选择明文攻击之外,采用这种简单的函数实际上没有解决任何问题。图 8-6 所示的协议仍然可以遭到并行会话攻击。攻击者只要在原来的消息上附加适当的偏值,就很容易将其攻破。具体攻击方法如图 8-7 所示。

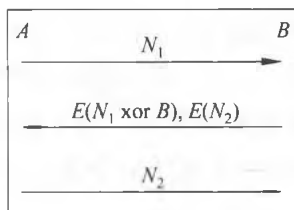


图 8-6 满足非对称要求的双向认证协议

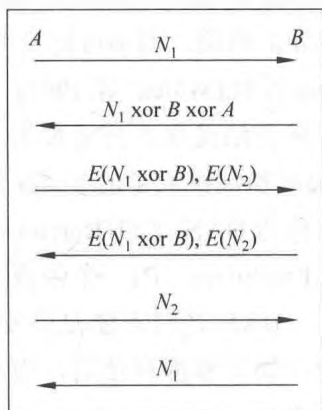


图 8-7 偏值攻击（通过并行会话攻击）

通过上面的讨论可以发现,图 8-2 中被认为十分安全的协议可以用多种方法攻破。在实际中,攻击者对一些看似安全的协议通过仔细地分析,可以发起诸如预言者会话攻击、并行会话攻击、偏值攻击或其他类型的攻击。这些攻击统称为交织攻击(Interleaving Attacks)。因此,如果所设计的协议是安全的,它就必须能够抵抗交织攻击。如何设计的协议才是安全的,这是许多协议设计者关心的问题。迄今为止,还没有更好、更系统的方法来设计安全的密码协议,这里只能将现有的一些最新研究成果介绍给读者。

8.5.2 密码协议的安全性分析

目前,对密码协议进行分析的方法有两种,一种是攻击检验方法,另一种是采用形式语言逻辑证明的方法。

1. 攻击检验方法

这种方法就是采用现有的一些有效的协议攻击方法,逐个对协议进行攻击,检验其是否具有抵御这些攻击的能力。分析时,主要采用语言描述的方法,对协议所交换的密码消息的功能进行剖析。

2. 形式语言逻辑分析法

采用形式语言对密码协议进行安全性分析的基本方法归纳起来有4种：

- (1) 采用非专门的说明语言和验证工具来对协议建立模型并加以验证。
- (2) 通过开发专家系统，对密码协议进行开发和研究。
- (3) 采用能够分析知识和信任的逻辑，对协议进行安全性研究。
- (4) 基于密码系统的代数特点，开发某种形式方法，对协议进行分析和验证。

第一种方法是將密码协议看成为计算机程序，并校验其正确性。然而，证明了正确性不等于证明了安全性。采用这一方法不能检测协议存在的安全缺陷。

第二种方法采用专家系统来确定协议是否能够达到某个不期望的状态。尽管这一方法能够很好地识别出存在的安全缺陷，但它不能保证安全性。它易于发现协议中是否存在某一已知的缺陷，而不可能发现未知的缺陷。这种方法的应用实例是美国军方开发的 Interrogator 系统[Millen 等 1987]。

第三种方法是迄今使用最为广泛的一种方法。美国 DEC 公司的 Michael Burrows, Matin Abadi 和剑桥大学的 Roger Needham 提出并开发了一个分析知识和信任的形式逻辑模型，称为 BAN 逻辑(Burrows et al 1989)。该逻辑假设认证是完整性(Integrity)和新鲜性(Freshness)的一个函数。在协议的整个运行过程中，采用逻辑规则来跟踪这两个属性。BAN 逻辑不能提供安全性证明，它只能用来推理认证。由于 BAN 逻辑简单、直观，便于掌握和使用，而且可以成功地发现协议中存在的安全缺陷，因此得到了广泛应用。

第四种分析密码协议的方法是将密码协议模型转化为一个代数系统，表述参与者对协议知识的状态，然后分析某种状态的可达性(Attainability)。然而，这种方法没有像 BAN 逻辑那样引起人们足够的重视。

目前，美国海军实验室的 NRL 协议分析器可能是这些技术最成功的应用。它被用来发现协议中可能存在的未知和已知的缺陷。此外，人们还尝试采用 NRL 协议分析器来设计密码协议。

在对缺少可信赖的第三方服务器实时参与的认证协议进行证明时，BAN 认证逻辑存在一定的局限性。为了便于对双钥体制构造的密码协议进行分析，不得不做出这样的假设：即由证书机构 CA 所颁发的证书均是新的(Fresh)。在做出这个最基本的假设之后，问题就可以得到解决了。在实际中，这一假设是合理的。因为证书的“新鲜性”可以由证书中所含的证书有效期以及作废证书清单 CRL 来保证[Aziz 等 1994]。

对于涉及 Diffie-Hellman 密钥交换系统的协议，BAN 逻辑无法对其进行分析。为了打破 BAN 逻辑的局限性，许多文献对 BAN 逻辑进行了某些必要的改进或扩展，这里将这些逻辑分别称为 GNY、AT、VO 和 SVO 逻辑。GNY 和 AT 逻辑对 BAN 进行了扩展，增加了许多逻辑规则以便更好地分析同一类协议。VO 逻辑则在 BAN 逻辑的基础上，增加了对 Diffie-Hellman 密钥交换系统的处理能力。而 SVO 逻辑则对以上 3 种逻辑进行了

归纳和总结,使其具有更加完善的形式分析能力。即便如此,将形式语言应用于密码协议分析仍然是一个全新的领域。

各种分析方法都有其特点和局限性,应综合分析利用。许多文献[Boyd 等 1993; Kemmerer 等 1994; Simmons 1994a, 1994b; Meadows 1994; van Oorschot 1993a; Gligor 等 1991]都讨论了这一理论的实用问题。

应当指出,通过攻击检验方法和形式逻辑分析法,都只是协议安全性的必要条件,而不是协议安全性的充分条件。攻击检验法可以验证协议有无已知类型的安全漏洞,不能证明能对付将出现的新的攻击方法;形式逻辑分析法在将协议以形式语言表述时,常常难以将所有可利用信息纳入,因而不能对协议进行完善的数学描述。这一方法还有待完善和发展。有关密码协议的设计规范和形式语言证明,请参考有关书籍和文献[Wang 等 1999]。

习 题

一、填空题

1. 计算机网络中使用的好的协议,不仅应该具有_____性、_____性和_____性,而且应该具有足够高的_____性。
2. 为有效地防止中间人攻击,在密钥交换协议中应采用_____技术。
3. 密钥建立协议主要分为_____协议和_____协议。
4. Diffie-Hellman 算法的安全性是基于_____。
5. 密钥认证分为_____、_____和_____三种。
6. 认证包含_____、_____和_____。
7. 认证的密钥建立协议有_____、_____、_____、_____、_____、_____、_____和_____。
8. 密码攻击的目标有_____、_____和_____。
9. 对协议进行攻击的典型方法有_____、_____、_____和_____。
10. 对密码协议的安全性进行分析的方法有_____和_____。

二、思考题

1. 什么是协议?协议具有哪些特点?协议有几种类型?
2. 什么是仲裁协议?仲裁协议有哪些特点?
3. 什么是裁决协议?裁决协议有哪些特点?
4. 什么是自执行协议?自执行协议有哪些特点?
5. 一个好的协议应具备哪些特点?
6. 按照密码协议的功能来分类,密码协议可以被分成哪几类?
7. 什么是中间人攻击?中间人攻击能够成功实施的真正原因是什么?

8. Diffie-Hellman 算法能否用来对消息进行加密和解密？为什么？

9. 掺杂是对付字典攻击的有效方法。请问它是否能抗击对单一口令的预定攻击？为什么？

10. 在密码协议中，一次性随机数（Nonce）和时戳（Timestamp）的作用是什么？

11. 对密码协议进行安全性分析有哪几种方法？